

Bloque 5 — Testbench Unitario

tb_rfft_recombine

Índice

1. Descripción General	2
2. Estructura del Testbench	2
3. Vectores de Prueba	4
4. Protocolo de Estímulo	4
4.1. Secuencia temporal	6
5. Verificador	6
5.1. Tolerancia	6
6. Comando de Simulación	7
7. Resultado	7
7.1. Casos de fallo detectados	7
8. Relación con Otros Testbenches	8

1. Descripción General

`tb_rfft_recombine` es el testbench unitario (nivel 12 de 13 en el plan de verificación) que valida el módulo `rfft_recombine` del Bloque 5 de forma aislada. Inyecta 1024 bins complejos $Z[k]$ (salida del Bloque 4) con el protocolo `fft_valid/fft_done`, captura los 512 bins reales $X[k]$ emitidos por la recombinación con decimación a bins pares, y los compara contra un modelo dorado (*golden*) generado por Python + NumPy.

Objetivo: verificar que la recombinación RFFT (ecuaciones de desempaquetado + butterfly con W_{2048}^k) es correcta a nivel de bit en Q15, con tolerancia controlada.

2. Estructura del Testbench

El testbench instancia tres componentes:

Cuadro 1: Componentes del testbench

Componente	Módulo	Rol
DUT	<code>rfft_recombine</code>	Módulo bajo prueba. Recibe $Z[k]$, emite $X[k]$ decimado.
Twiddle ROM	<code>twiddle_rom</code> (B3)	Provee W_{2048}^k a través del puerto <code>tw_addr/data_recomb</code> . Inicializada con <code>twiddles_recomb.hex</code> .
Verificador	Lógica en el <code>always</code> del TB	Compara cada <code>g_valid</code> contra el golden almacenado en <code>x_gold[0..511]</code> . Acumula errores.

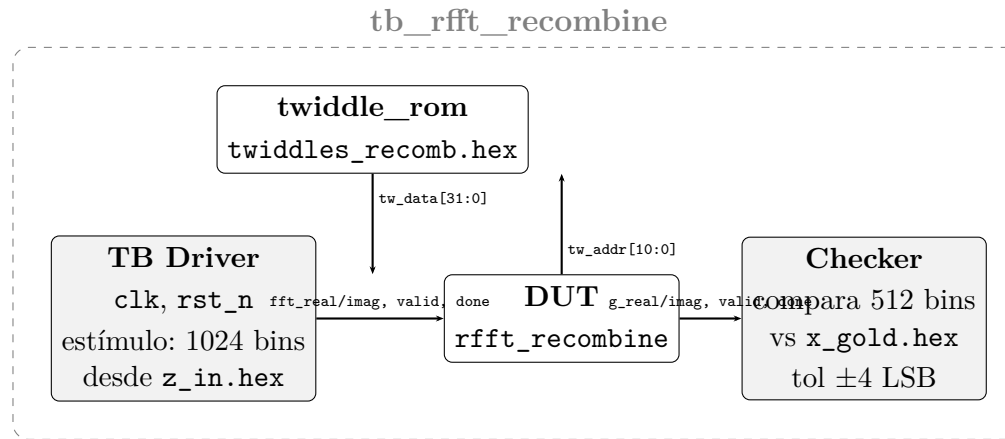


Figura 1: Diagrama de bloques del testbench. El driver inyecta los 1024 bins $Z[k]$ desde `recomb_z_in.hex`. El DUT consulta la ROM de twiddles de recombinação y emite 512 bins reales. El checker compara cada bin contra `recomb_x_gold.hex` con tolerancia ± 4 LSB.

3. Vectores de Prueba

Los vectores son generados por `scripts/gen_e2e_vectors.py` usando el modelo dorado de Python con NumPy:

```
import numpy as np

def rfft_golden(x):
    return np.fft.rfft(x, n=2048) / 1024
```

Se generan dos archivos `.hex` (formato `$readmemh` de Verilog, un valor de 32 bits por línea):

Cuadro 2: Archivos de vectores de prueba

Archivo	Entradas	Formato	Contenido
tb/vectors/ recomb_z_in.hex	1024	{real[15:0], imag[15:0]}	$Z[k]$: FFT compleja del tono empaquetado, escalada 1/1024. Formato Q15, 32 bits por línea.
tb/vectors/ recomb_x_gold.hex	512	{real[15:0], imag[15:0]}	$X[k]$: golden esperado (solo bins pares $k = 0, 2, \dots, 1022$). Parte imaginaria ≈ 0 (espectro real).

Tono de prueba: frecuencia 3 kHz, $f_s = 48$ kHz, $N = 2048$, amplitud 0,5 en Q15. Esto produce un pico en el bin $k = \lfloor 3000 \times 2048 / 48000 \rfloor = 128$ del espectro completo, que corresponde al bin de display $d = k/2 = 64$.

4. Protocolo de Estímulo

El driver del testbench sigue la secuencia de la Figura 2:

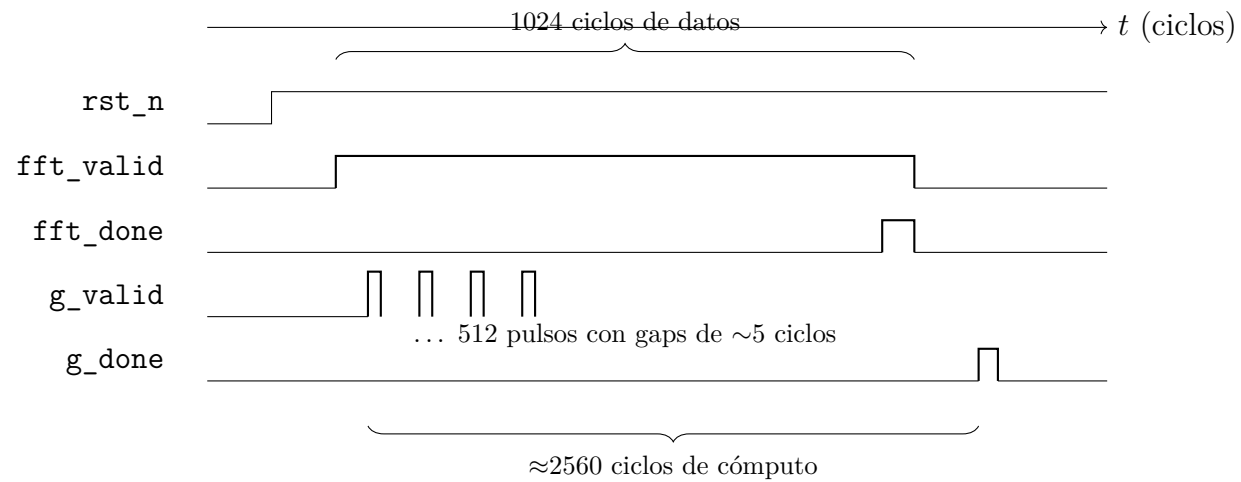


Figura 2: Diagrama de tiempos simplificado del testbench. El reset se libera tras 5 ciclos. Los 1024 bins $Z[k]$ se envían con **fft_valid** continuo; **fft_done** se activa simultáneamente con el último **fft_valid**. La salida **g_valid** emite 512 pulsos espaciados ~ 5 ciclos (por la latencia de la FSM). Finalmente **g_done** marca el fin del cómputo.

4.1. Secuencia temporal

```

1. rst_n = 0 durante 5 ciclos
2. rst_n = 1, esperar 5 ciclos
3. for i = 0..1023:
    @(posedge clk)
    fft_real <= z_in[i][31:16]
    fft_imag <= z_in[i][15:0]
    fft_valid <= 1
    fft_done <= (i == 1023)
4. @(posedge clk): fft_valid=0, fft_done=0
5. Esperar g_done con timeout de 20000 ciclos
6. Verificar: out_cnt == 512, done_seen == true, errors == 0

```

5. Verificador

El verificador opera en el flanco positivo de clk, muestreando cada g_valid:

```

1 always @(posedge clk) begin
2     if (rst_n && g_valid) begin
3         got_r = $signed(g_real);
4         got_i = $signed(g_imag);
5         exp_r = $signed(x_gold[out_cnt][31:16]);
6         exp_i = $signed(x_gold[out_cnt][15:0]);
7         er = (got_r > exp_r) ? got_r - exp_r : exp_r - got_r;
8         ei = (got_i > exp_i) ? got_i - exp_i : exp_i - got_i;
9         if (er > TOL || ei > TOL) begin
10             if (errors < 16)
11                 $display("FAIL bin %0d: got (%0d,%0d) exp (%0d,%0d)",
12                     out_cnt, got_r, got_i, exp_r, exp_i);
13             errors = errors + 1;
14         end
15         out_cnt = out_cnt + 1;
16     end
17     if (rst_n && g_done) done_seen = 1'b1;
18 end

```

5.1. Tolerancia

Cuadro 3: Tolerancia del verificador

Parámetro	Valor	Justificación
TOL	± 4 LSB	Acumula: ± 1 LSB de cuantización de la ROM de twiddles W_{2048}^k , ± 2 LSB del butterfly (4 productos Q15 con redondeo), ± 1 LSB de margen.
Errores tolerables	0	Test unitario bit-exacto con modelo dorado determinista. Cualquier discrepancia > 4 LSB indica un bug en el RTL.

Se muestran hasta 16 errores de detalle antes de abortar la impresión para no saturar la salida de simulación.

6. Comando de Simulación

El testbench se ejecuta con Icarus Verilog 12.0 desde el directorio `final/` (necesario para que `$readmemh` resuelva las rutas relativas de los archivos `.hex`):

```
cd final/

iverilog -g2012 -o tb_rec tb/tb_rfft_recombine.v \
  src/block5/rfft_recombine.v \
  src/block3/butterfly_radix2.v \
  src/block3/twiddle_rom.v

vvp tb_rec
```

Dependencias:

- `src/block3/twiddles_fft.hex` — ROM de twiddles FFT (puerto no usado en este TB, pero requerido por la instanciación de `twiddle_rom`).
- `src/block3/twiddles_recomb.hex` — ROM de twiddles de recombinación (1025×32 bits, W_{2048}^k).
- `tb/vectors/recomb_z_in.hex` — 1024 bins complejos de entrada.
- `tb/vectors/recomb_x_gold.hex` — 512 bins golden esperados.

7. Resultado

El testbench produce una salida binaria (PASS/FAIL). La ejecución exitosa se ve así:

```
TB RECOMBINE: PASS (512 bins vs golden, tol +-4 LSB)
tb/tb_rfft_recombine.v:132: $finish called at 71910000 (1ps)
```

El tiempo de simulación es de aproximadamente $71.91 \mu s$ ($7,191,000 \text{ ns} = 7,191$ ciclos de reloj a 50 MHz de simulación), consistente con:

$$\text{ciclos totales} = 1024_{\text{captura}} + 512 \times 5_{\text{FSM}} + \text{overhead} \approx 3584 + \text{margen}$$

7.1. Casos de fallo detectados

El verificador captura los siguientes escenarios de error:

1. **Conteo incorrecto de bins:** si `out_cnt` $\neq$ 512 al finalizar, se reporta **FAIL**: se esperaban 512 bins, llegaron N.
2. **g_done ausente:** si la señal nunca se activa dentro del timeout de 20,000 ciclos, se reporta **FAIL**: `g_done` nunca llevo.
3. **Error de magnitud por bin:** cada bin cuya diferencia con el golden exceda ± 4 LSB incrementa `errors`. Si `errors` > 0 al final, se reporta **FAIL**, `errores=N`.

8. Relación con Otros Testbenches

El plan de verificación del proyecto incluye 13 testbenches. El `tb_rfft_recombine` ocupa la posición #12:

Cuadro 4: Testbenches relacionados con el Bloque 5

#	Testbench	Alcance	Estado
11	<code>tb_chain_b2b4recomb</code>	Cadena B2→B4→recomb vs. golden	PASS
12	<code>tb_rfft_recombine</code>	Unitario de recombina- ción (512 bins, ± 4 LSB)	PASS
13	<code>tb_rfft_scope_e2e</code>	End-to-end: UART (tono 3 kHz) → píxeles LCD	PASS

El TB de cadena (#11) valida la integración B2–B4–recomb con datos provenientes del bloque de permutación. El TB unitario (#12) aísla únicamente la recombinación, inyectando $Z[k]$ directamente y verificando $X[k]$ contra el golden. El TB end-to-end (#13) cierra el lazo completo desde la entrada UART hasta los píxeles del LCD.

Nota: los módulos `spectrum_buffer` y `spectrum_draw` no tienen testbench unitario independiente. Se verifican exclusivamente en el TB end-to-end (#13), donde se comprueba que los píxeles generados para un tono de 3 kHz coinciden con la posición esperada en el display (± 2 px de tolerancia visual).